Mini AI project 보고서

SKALA 4기 광주캠퍼스 2반 우상민

Chapter 1. How it works

1. 수행방법

All_total_vue_student.zip 의 코드를 chat GPT에게 제공한 후

모든 코드를 이해할 수 있도록 완전 0(no git, no code, no venv)상태에서 단계별로 코드를

빌드해서 최종적으로 예시코드와 같은 기능을 구성한 후

그 코드로부터 발전시킬 부분을 발전시키기로 결정하였다.

2. 기본 코드 구성 진행

1 - monitoring.py

```
(.venv) wsm@skalawsmui-MacBookPro ~/workspace/ai-minip python step01_monitoring.py
조회할 서비스 이름: payments-service
서비스: payments-service
상태: degraded
CPU: 91.0%
Memory: 55.0%
알림: ['CPU 91% (threshold 80%)', 'payments-service-3 instance responding slow']
(.venv) wsm@skalawsmui-MacBookPro ~/workspace/ai-minip
```

Monitoring.json으로부터 이름을 key 값으로 모니터링 mock data를 가져온다는것을 확인했다.

2 - diagnosis.py

```
(.venv) wsm@skalawsmui-MacBookPro ~/workspace/ai-minip  main ± python step02_diagnosis.py
진단할 서비스 이름: payment-service
등록되지 않은 서비스입니다.
(.venv) wsm@skalawsmui-MacBookPro ~/workspace/ai-minip  main ± python step02_diagnosis.py
진단할 서비스 이름: payments-service
payments-service: 이상 상태(degraded)입니다.
- CPU 91% (threshold 80%)
- payments-service-3 instance responding slow
(.venv) wsm@skalawsmui-MacBookPro ~/workspace/ai-minip  main ± python step02_diagnosis.py
진단할 서비스 이름: web-01
web-01: 정상입니다.
```

크게 새로운 변화는 없었다. 1에서 파일입출력과 결과판단의 코드를 모듈로 분리했다.

3 - langgraph.py

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

from step02_diagnosis import diagnose_service


# 공유 상태 정의
class AgentState(TypedDict):
    system_name: str
    diagnosis: str


# 진단 노드
def diagnosis_node(state: AgentState):
    result = diagnose_service(state["system_name"])
    return {"diagnosis": result}


# 그래프 구성
builder = StateGraph(AgentState)

builder.add_node("diagnosis", diagnosis_node)

builder.add_edge(START, "diagnosis")
builder.add_edge("diagnosis", END)

graph = builder.compile()


# 실행
if __name__ == "__main__":
    system_name = input("진단할 서비스 이름: ")

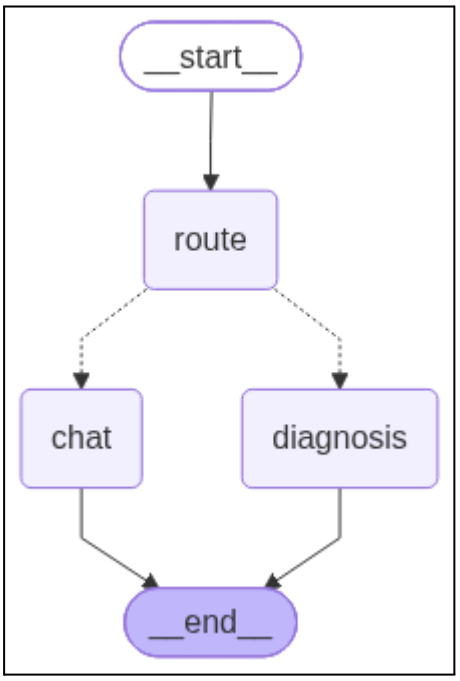
    result = graph.invoke({"system_name": system_name})

    print(result["diagnosis"])
```

Llm 연결 없이 diagnosis 함수 단 하나를 langgraph 노드에 연결해 보았다.

함수가 노드에 들어갔기 때문에 2번과 입출력이 동일하다

4 - routing.py



분기노드(conditional_edges)를 추가해 보았다.

요청분류노드 route_node 에서 State값을 한번 지정해주고
실행경로선택 select_route 함수를 작성해서 여기서도 State를 읽어서 다음 실행 경로를 결정하는값을 따로 반환할 수 있다. (!!)
(원본코드의
def _after_route(state: AgentState) -> str:
    return "chat_answer" if state["route"] == "chat" else "plan"
이런 느낌) 최종 경로는 add_conditional_edges의 dict에서 반환값을 실제 Node에 매핑

after route처럼 복잡하게 분기하는경우 사용할 수 있겠음
그래프에서 봤을때는 그냥 하나의 route node에서 갈라지는것처럼 보임

복잡하지 않은 경우 따로 함수를 작성하지 않고 lambda로 처리할 수 있음

5 - llm_routing.py

```python
# LLM 요청 분류 노드
def route_node(state: AgentState):
    messages = [
        ("system",
        "당신은 IT 서비스 운영 요청 분류기입니다.\n"
        "사용자 요청을 diagnosis 또는 chat으로 분류하세요.\n"
        "diagnosis: 장애 의심, 성능 저하, 응답 지연, 모니터링, 상태 확인\n"
        "chat: 인사, 잡담, IT 운영과 무관한 질문\n"
        "서비스 이름이 없어도 장애나 성능 문제를 언급하면 diagnosis입니다.\n"
        "예시:\n"
        "'서버가 느린 것 같아' → diagnosis\n"
        "'CPU가 너무 높아' → diagnosis\n"
        "'payments-service 확인해줘' → diagnosis\n"
        "'안녕하세요' → chat\n"
        "사용자가 명시한 서비스 이름만 추출하세요.\n"
        "서비스 이름이 없으면 빈 문자열을 사용하세요.\n"
        '반드시 {"intent": "...", "system_name": "..."} 형식의 JSON만 반환하세요.'),
        ("human", state["user_input"])
    ]

    response = llm.invoke(messages)
    data = json.loads(response.content)

    # LLM 분류 결과 검증
    if data.get("intent") not in ("diagnosis", "chat"):
        raise ValueError("LLM이 올바르지 않은 분류값을 반환했습니다.")

    system_name = data.get("system_name", "")

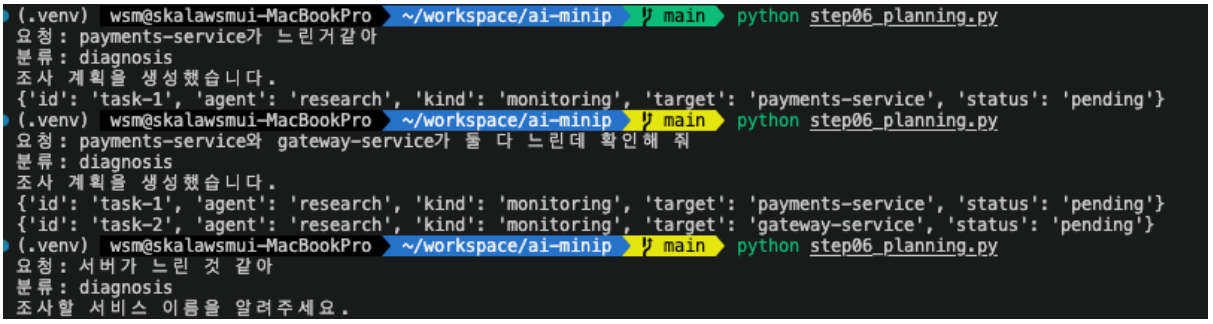
    if not isinstance(system_name, str) or system_name not in state["user_input"]:
        system_name = ""

    return {
        "intent": data["intent"],
        "system_name": system_name
    }
```

Route_node를 일반 코드분기에서 llm이 state를 입력하도록 바꾸었다

특히 ollama qwen 모델을 3.5:9b 로 업그레이드했는데
같은 프롬프트라도 이 분류 성능자체가 이정도 모델은 되어야 정상작동한다는것을 확인했다.

6 - planning.py



원본 코드의 planning을 단순화시켜서 사용해보았다
Planning (json 반환) 이 잘 진행되는것을 확인했다.

7 - research node

def research_node(state: AgentState):
tasks = [task.copy() for task in state["tasks"]]
collected = []

이 코드가 핵심이라고 생각한다. Task에서 작업 실행상태를 관리하고, Collected가 실제 조회 결과를 보관한다 → 따라서 이 정보를 다시 llm에 넘기면 정보의 충분함 판단 및 Replan 판단이 가능해 질 것이다.

8 - MCP (9에서 이어서 계속)

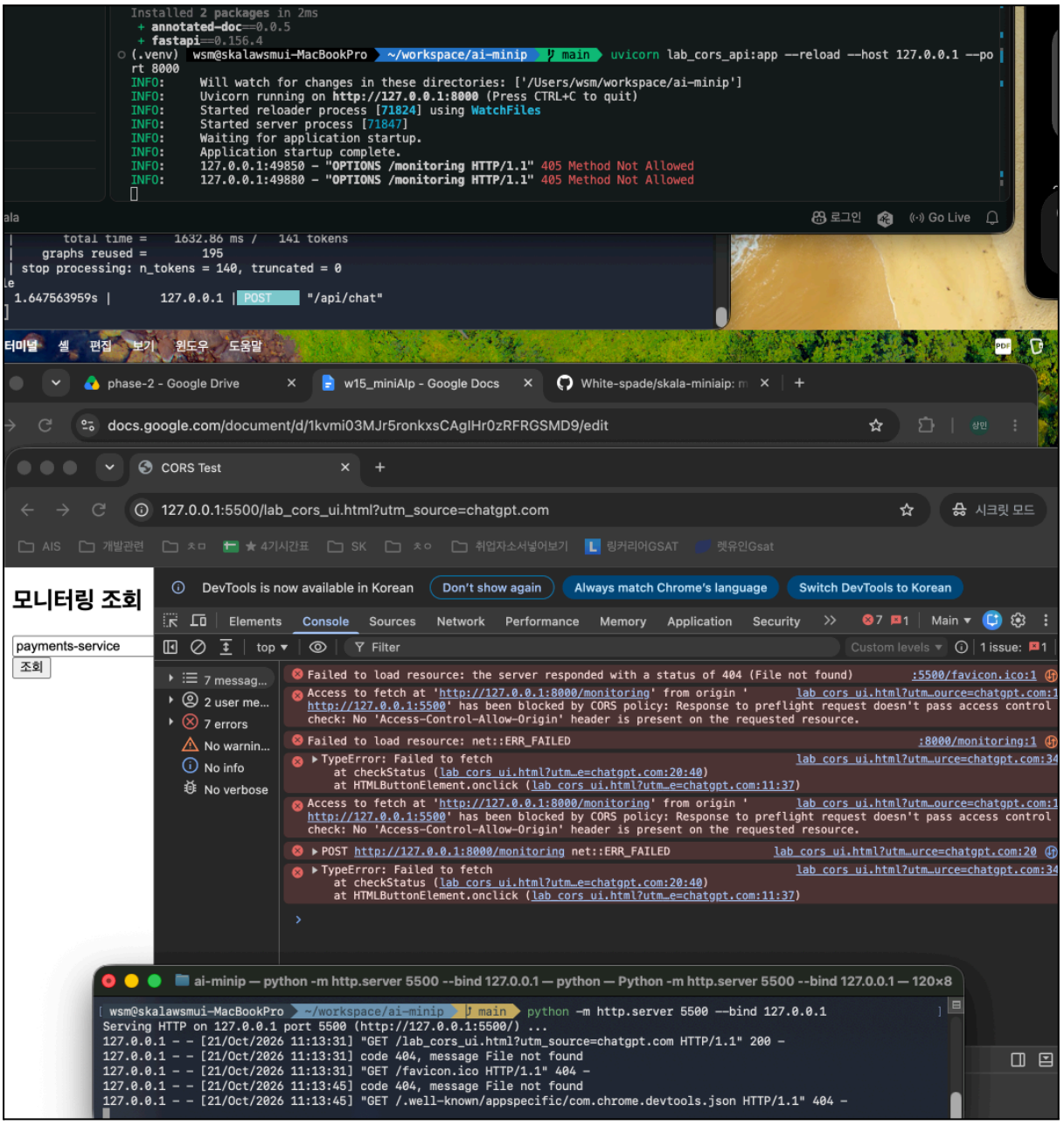
7번에서 data = get_monitoring_status(task["target"])로 직접호출하던 함수가

다음과 같이 변경된다 (실제 API는 구현마다 다르지만, 입력형식과 이름/반환만 알면 됨)

```
data = mcp_client.call_tool(
 "get_monitoring_status",
 {"system_name": task["target"]}
)
```

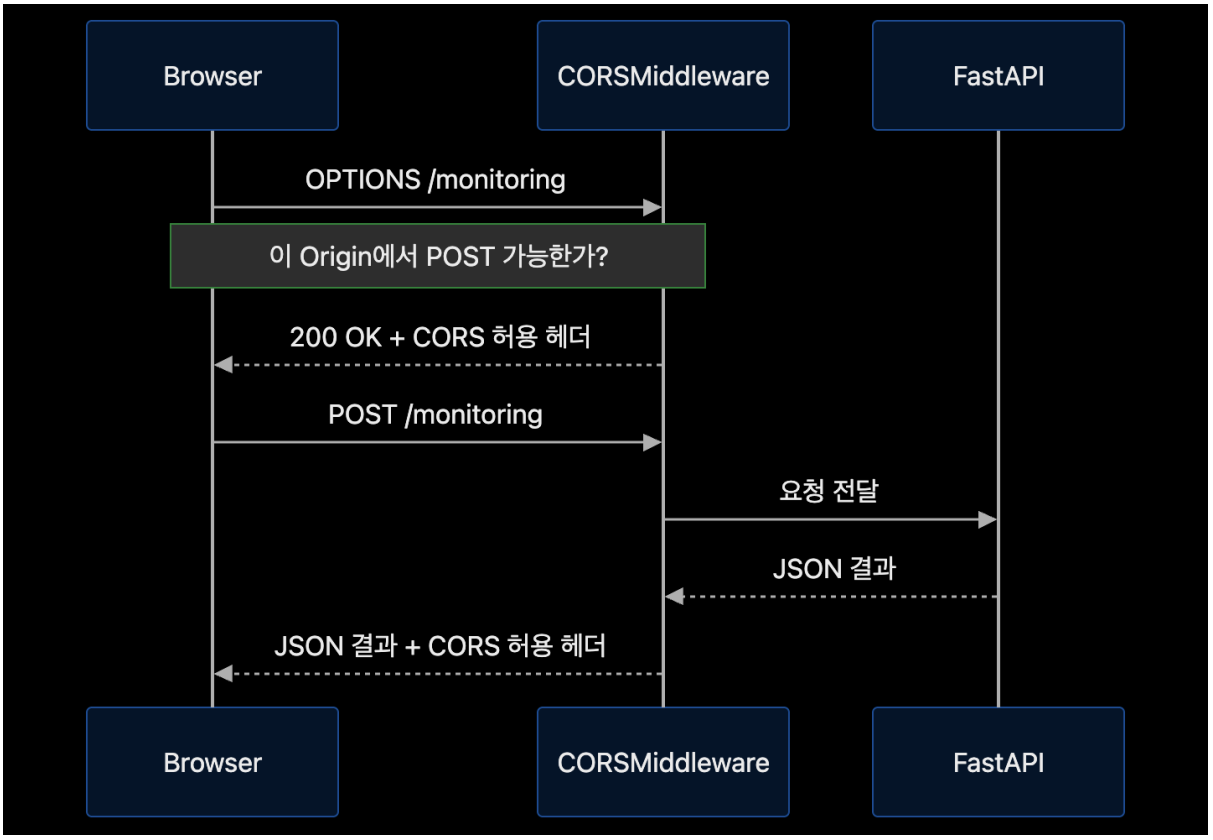
8-2 - CORS 상황과 Fastapi middleware

교수님의 원본코드는 Origin을 맞춰놔서 이런 문제가 뜰 일이 없다는 설명을 들었다.
Origin은 Protocol + Host + Port의 조합을 의미하고 로컬에서 다른 port로부터 UI(front)와 데이터가 들어온다면 CORS 에러가 뜰 것이다.
해당 에러를 시연하고 middleware로 고쳐보도록 한다.



fastapi는 8000포트인데 html서버는 5500포트라서 CORS 에러가 난 상황이다

Fast api 측에 add.middleware로 allow origin을 추가해 해결해보았다.



이 경우에 browser는 post 요청 전 preflight로 허용가능한 origin인지 확인을 받는다.
포트(origin)가 다르다는것 자체가 오류가 아니라 CORS 검사가 필요해 진 상황으로 이해했다.

```
app.add_middleware(
 CORSMiddleware,
 allow_origins=["http://127.0.0.1:5500"],
 allow_methods=["POST"],
 allow_headers=["Content-Type"]
)
```

추가 후 화면은 정상적으로 출력된다 (캡처는 생략)
이게 필요한 이유는 웹 제공 서버랑 ai serve 서버의 origin이 다르기 때문일 것이다.
실제로 vue.js (Vercel)을 통해서 입력한다고 하면
AI 서버쪽 fastapi에 CORS middleware - 허용으로 CORS오류를 해결할 수 있다.

9 - mcp client

이해가 안갔던 부분 1 - stdio로 구현되어있는 server-client 구조

원래 알고있는 서버-클라이언트는 서버가 먼저 실행되어 기다리다가 클라이언트의 요청을 처리하는 방식이었다.

그런데 그게아니라 stdin/stdout 통신으로 서버-클라이언트 구현을하니까
밑의 클라이언트의 코드가 서버를 execute해서 답변을 받아온다. 그부분이 헷갈렸다.

```
params = StdioServerParameters(
    command=sys.executable,
    args=[str(server_path)]
)
```

이해가 안갔던 부분 2 - 같은 로컬데이터, 디렉토리에 있어서 기능 주체가 헷갈렸다.

서버에서 from step01_monitoring import get_monitoring_status as load_status 얘를 가져와서 load status 함수를 실행할 때 서버측에서 데이터를 읽었다
그리고 클라이언트측에서는 tool call 요청, 즉

```
response = await session.call_tool(
 "get_monitoring_status",
 arguments={"system_name": system_name}
 )
```

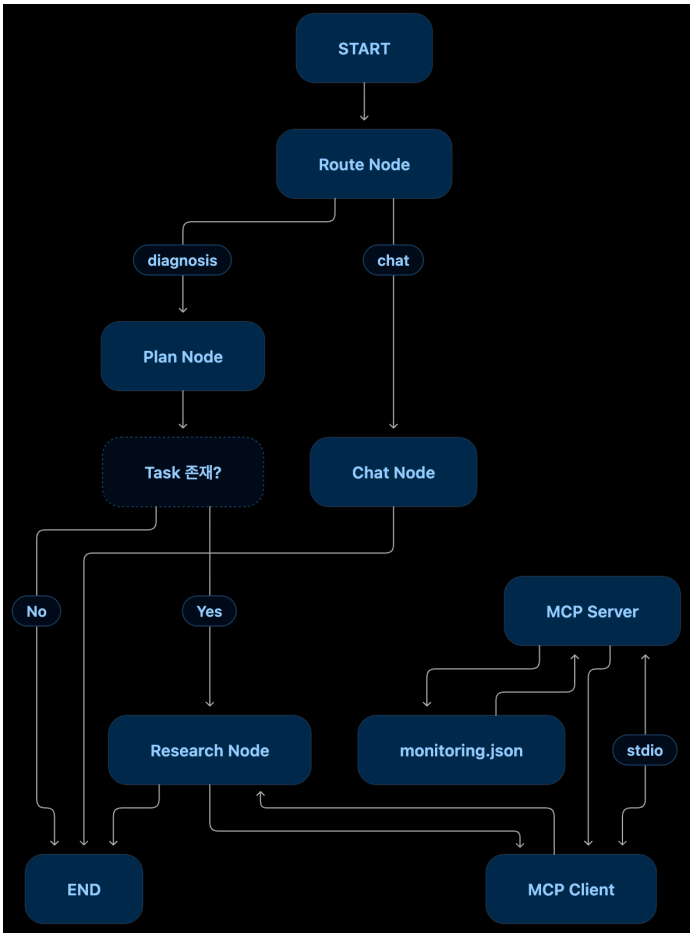
이게 실행되어 결과를 받아온것이다.

10 - Langgraph (research node) with MCP

노드 함수가 동기함수에서 비동기 함수가 되었다.

async def research_node(state: AgentState): …

response = await session.call_tool(
 "get_monitoring_status",
 arguments={"system_name": task["target"]}
)
data = json.loads(response.content[0].text)



아키텍처의 복잡도가 상승했다. 하지만 여전히 Langgraph 자체는 step7의 그것과 같다.
현재코드에서는 두개이상의 요청이 진행되고있지는 않기때문에 비동기실행의 이점을 받고있지 않지만, mcp sdk의 init, call_tool 등이 비동기(await session …) 설계되어있기 때문에 사용이 요구된다.

11 - write node : llm이 조회결과를 해석하고 설명하도록 한다.

route나 plan과 다르게 init llm할 때 format = "json" 매개변수가 없다.
자연어처리를 할것이기 때문이다.



`collected` 데이터를 기반으로 보고서를 작성하고 프롬프트 형식대로 출력된것을 확인했다.

12 - Runbook

기존 MCP서버에 더해 resource() (runbook)를 추가했다.

uri = f"runbook://incident-response/{incident_type}"

resource = await session.read_resource(uri)

를 통해 mcp에서 문서(resource)를 조회했다.



Workflow대로 runbook을 조회하고 기반으로 답변하는것을 확인했다.

다만 'payments-service의 응답 지연이 gateway-service에서 504 타임아웃과 패킷 손실로 이어진 것으로 보입니다.' 같은 확실하지않은(유추할 논리는 있지만 runbook에 존재하지않음) 결과를 제공하는것에 대해 validation node의 필요성도 확인할 수 있었다.

13 - MCP template

Langchain에서 promptTemplate로 사용하던걸 MCP서버에 저장해서 불러오는 방식이다.

```
prompt = template.format(
    system_name="payments-service",
    symptom="CPU 급증"
)# 이렇게 사용하던걸
response = await session.get_prompt(
 "incident_analysis_prompt",
 arguments={
 "system_name": "payments-service",
 "symptom": "CPU 급증"
 }
)# 이렇게 바꿈, MCP 서버 내부에서 PromptTemplate사용 가능
```

14 - gate Node

LLM이 작업을 생성할 때 잘못된 작업이나 모순된 판단을 내리는 경우, ex) 원본코드에서

needs_analysis = False

tasks = [{"agent": "research", "kind": "monitoring"},

{"agent": "analysis", "kind": "pattern-analysis"}]

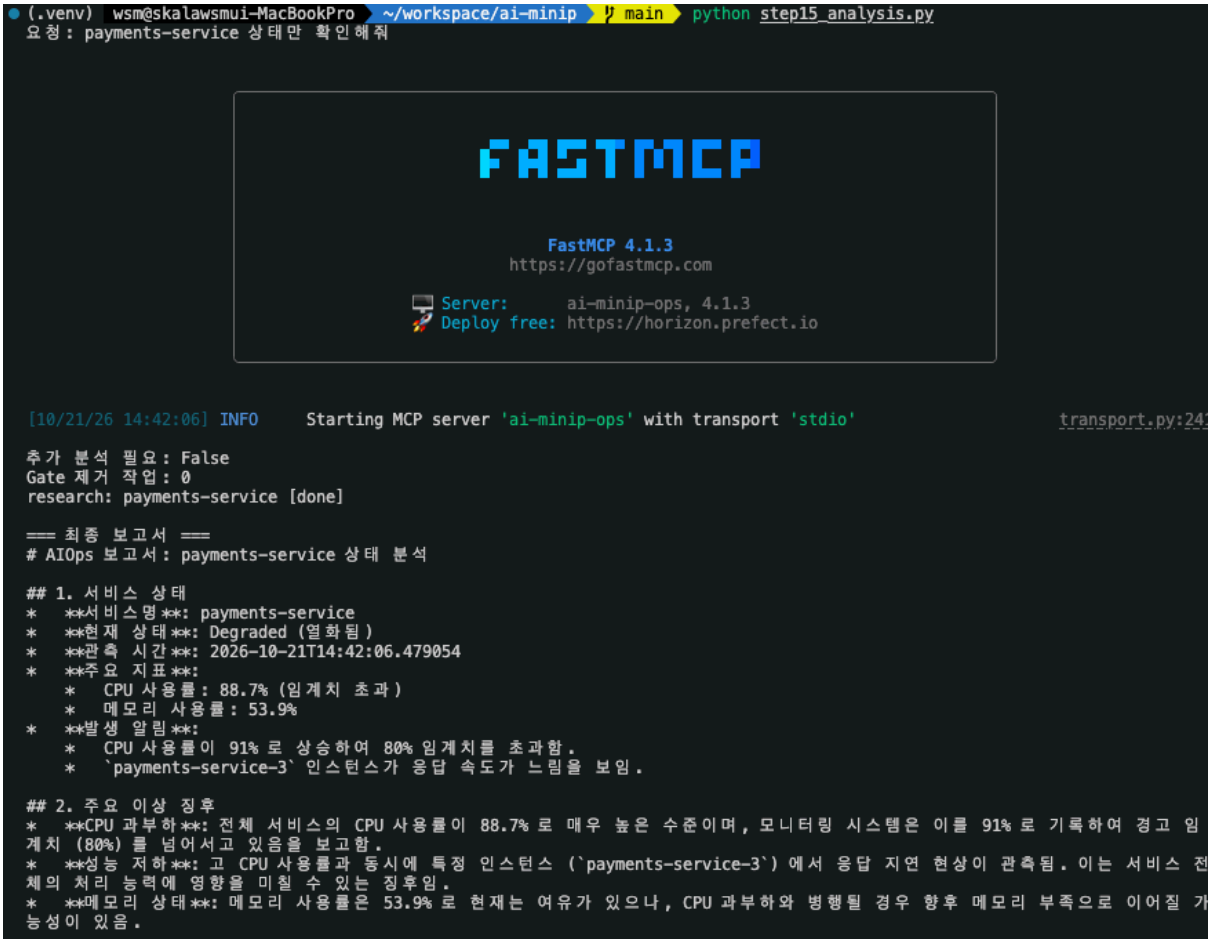
이러한 상황일 때 다시 판단을 llm에 맡기는 대신 코드로 실행을 통제하는 방식이다.

구현 시 노드가 async가 아님에도 주의한다 (외부응답을 기다리지 않으므로)

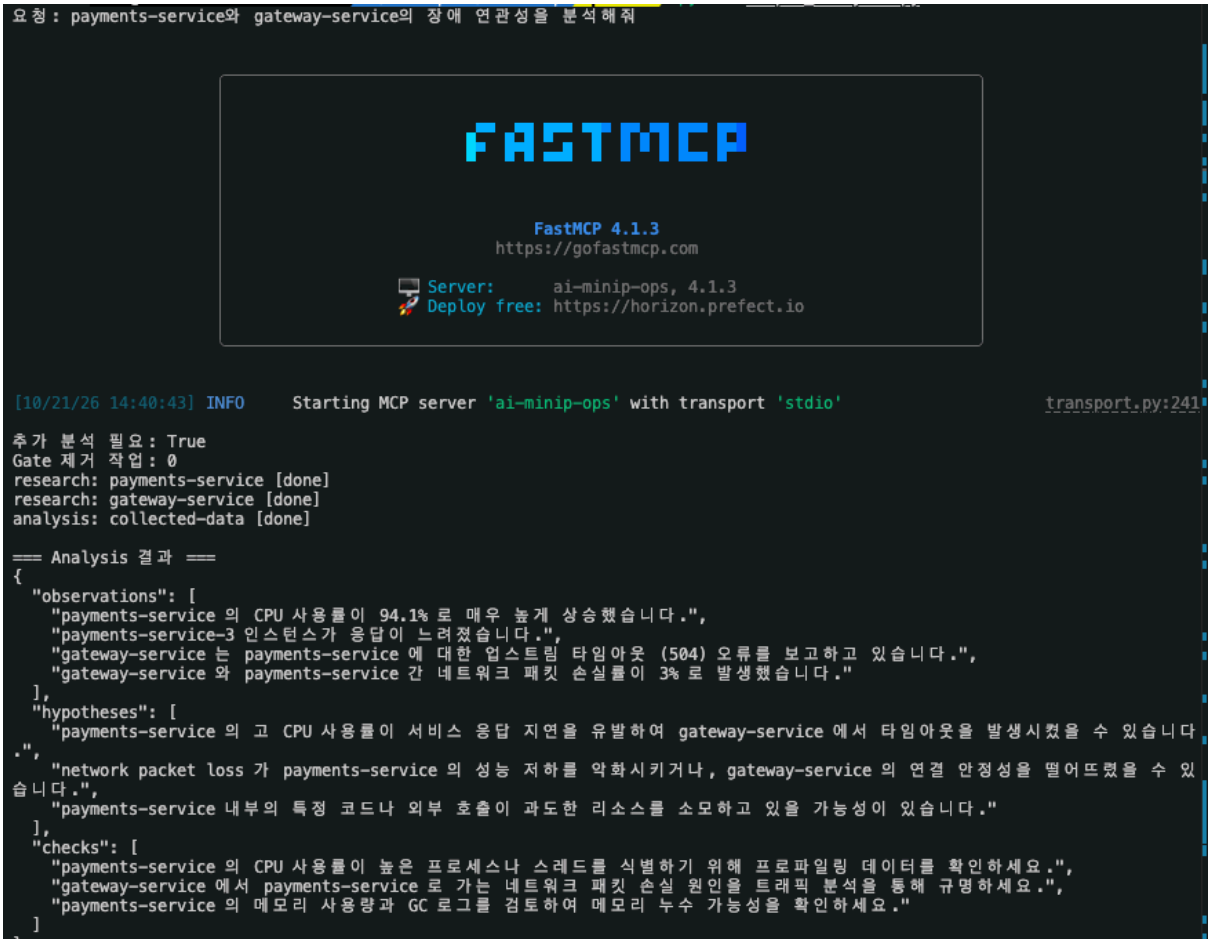
* *def gate_node(state: AgentState):*

15 - Analysis node

지금까지는 Write Node가 수집된 데이터로부터 '장애 원인을 추론'하고 '최종 사용자 보고서 작성' 두가지 일을 한번에 하고있었다. 분석, 보고서작성 두가지 역할을 분리한다.



상태만 확인해달라고 했을때는 analysis 가 진행되지 않은것을 확인할 수 있다.



분석해달라고 했을때는 analysis 후 결과가 state를 타고 최종 write(보고서작성)에 반영되는것을 확인할 수 있었다.

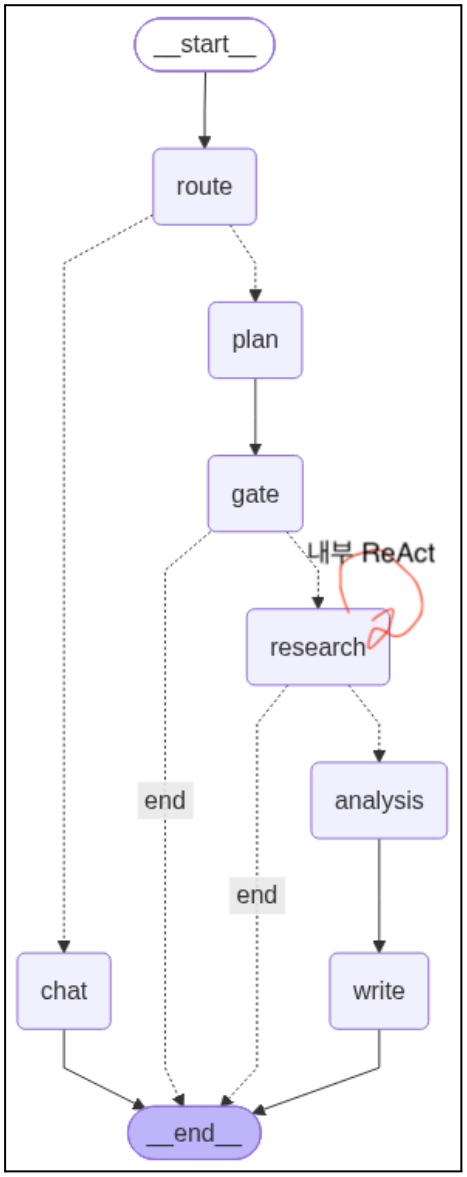
16 - ReAct (Plan과 ReAct의 차이?)
Plan은 사용자 요청으로부터 LLM이 계획을 생성하고 task를 진행하는것을 의미한다.

ReAct(Reasoning + Acting) 는 tool등의 하나의 작업 수행 이후 결과를 확인하고 재판단하는 것을 의미한다.
둘은 배타적인관계가 아니라, Plan은 전체 조사 방향을 설정하고, ReAct는 그 계획을 수행하면서 구체적인 Tool 호출을 결정하도록 결합하는 방식으로 사용할 수 있다.

이전에 나는 두 작업의 차이가 재작업의 유무라고 생각했다. 반쪽짜리 정답이었다.

이번 코드에서는 StateGraph로 직접 ReAct 반복구조를 작성했지만
LangGraph에서는 이를 자동 구성하도록 하는 기능도 존재한다는 것을 확인했다.
이 자동구성이 자주사용하는 create_agent() 이다.

현재 ReAct는 research_node 내부에서
'ReAct → ToolNode → ReAct → ToolNode → … → END' 하도록 구성했다.
따라서 외부그래프에서는 평범하게 'Route → Plan → Gate → Research → Analysis → Write' 로 보인다. 정식 Subgraph Node로 직접등록 할 수도 있다.(Create_agent와는 다른 방식이다.)



17 - FastAPI 통합, Validation, Exception, CORS 적용



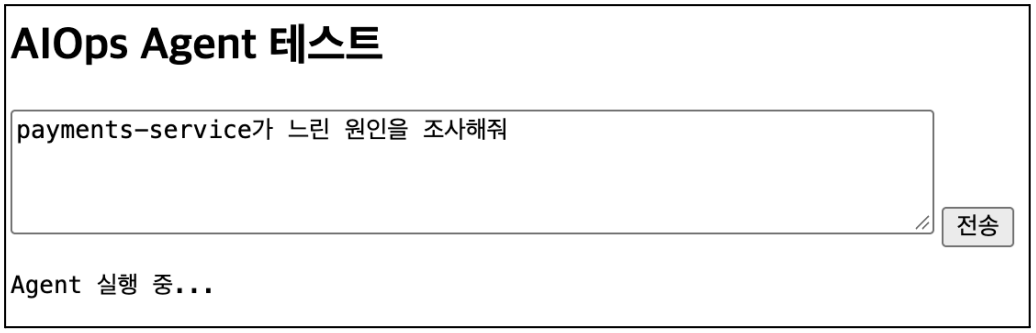
fastAPI에 올렸다. 정상응답을 확인했다.

from step16_react import graph

try:
LangGraph 실행
state = await graph.ainvoke({
"user_input": request.query
})

이렇게 간단한 코드로 구성해놓은 langgraph를 가져올 수 있었다,

Html도 step08에서 구성한것과 거의 동일한것을 가져왔다.
이번에도 포트가 5500과 8000으로 다르지만, middleware설정을 해 놓았기 때문에 문제없이 가져올 수 있다. 로딩화면도 정상작동했다.



18 - Replan & Validation 보고서 품질의 평가와 반복수행

ReAct랑 Replan은 또 뭐가다른가?
ReAct → 다음에 실행할 Tool 선택(결과 관찰), Replan → 결과가 부족하면 새로운 Task 생성
(+Validate → 보고서에 문제가있으면 재작성 요청)

예를들어 현재 프로젝트에서는 다음과 같다.
ReAct: gateway-service를 조회했더니 payments-service에서 타임아웃이 발생(관찰)합니다. payments-service도 조회할지 판단합니다.

Replan: Research가 끝난 후 조사 결과를 전체적으로 살펴봅니다. 추가로 조회해야 할 서비스가 남아 있다면 새로운 Task를 만들어 Research를 다시 실행합니다.

Validate: 최종 보고서에 관측되지 않은 수치가 등장하거나 원인을 과도하게 단정했다면 Write Node에 수정을 요청합니다.

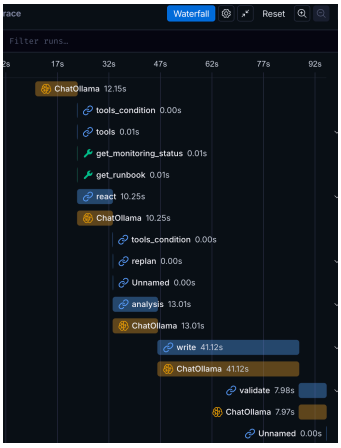
이러한 재작업 요청에는 항상 timeout과 최대 재작업 수치를 명시해야함에 주의.

19 - LangSmith

```
from langsmith import traceable
from step18_replan_validate import graph

@traceable(name="aiops_chat_request")
async def run_agent(query: str):
 return await graph.ainvoke({"user_input": query})
```

이렇게 간단하게 AOP관점에서 그래프 추적이 가능하다.



Write에 아주 오랜시간이 걸리는걸 추적해보았다.

20 - Service Validation + Clarification

Agent가 등록되지 않은 서비스를 임의로 조사하거나, 대상이 불명확한 요청을 추측해서 실행하지 않도록 개선하는것을 목표로 두었다.

서비스 검증 - `monitoring.json`에 등록된 서비스인지 확인
Hallucination 방지 - LLM이 임의로 생성한 조사 대상 차단
Clarification - 서비스가 불명확하면 재질문
후보 선택 - 서비스명을 직접 입력하거나 번호로 선택
재질문 제한 - 잘못된 답변이 3회 누적되면 요청 취소
후속 요청 처리 - 이전 요청과 사용자의 답변을 결합

여기에 다음 21번에서는 상태를 SQLite로 저장하려고하는데, 아직 구현이 안되었으니
20번에서는 Python on memory 대기를 잠시 유지하도록 한다.

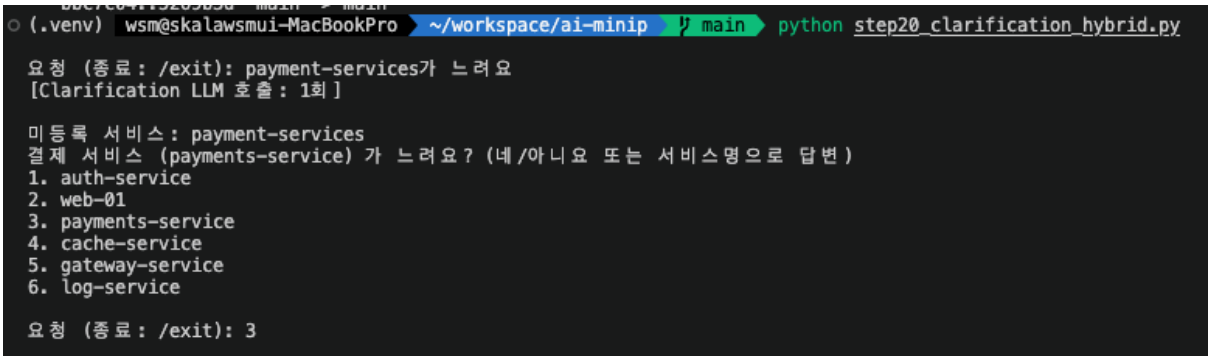
```
def prepare_node(state: AgentState):
 pending = state.get("pending_query", "")
 if not pending:
 return {"next_step": "route"}
```

재 질문을 위해 pending_query string state를 추가했다.

```
def mentioned_services(text: str) -> list[str]:
 return [name for name in SERVICES if re.search(
 rf"(?<![A-Za-z0-9_-]){re.escape(name)}(?![A-Za-z0-9_-])", text
 )]
def unknown_services(text: str) -> list[str]:
 # 서비스명처럼 보이지만 목록에는 없는 이름 탐지
 pattern =
r"(?<![A-Za-z0-9_-])(?:[A-Za-z][A-Za-z0-9-]*-service|web-\d+)(?![A-Za-z0-9_-])"
 return [name for name in set(re.findall(pattern, text)) if name not in SERVICES]
```

서비스명과 아닌것을 정규식으로 일차로 판독하는 코드도 추가되었다.

또 기존 코드의 구조에서 질문이 부정확하면 다시물어보는걸 판독하는 route를 하나 붙이고 재질문 true 인경우 질문을 작성하는 llm을 호출하도록 graph를 업데이트했다.



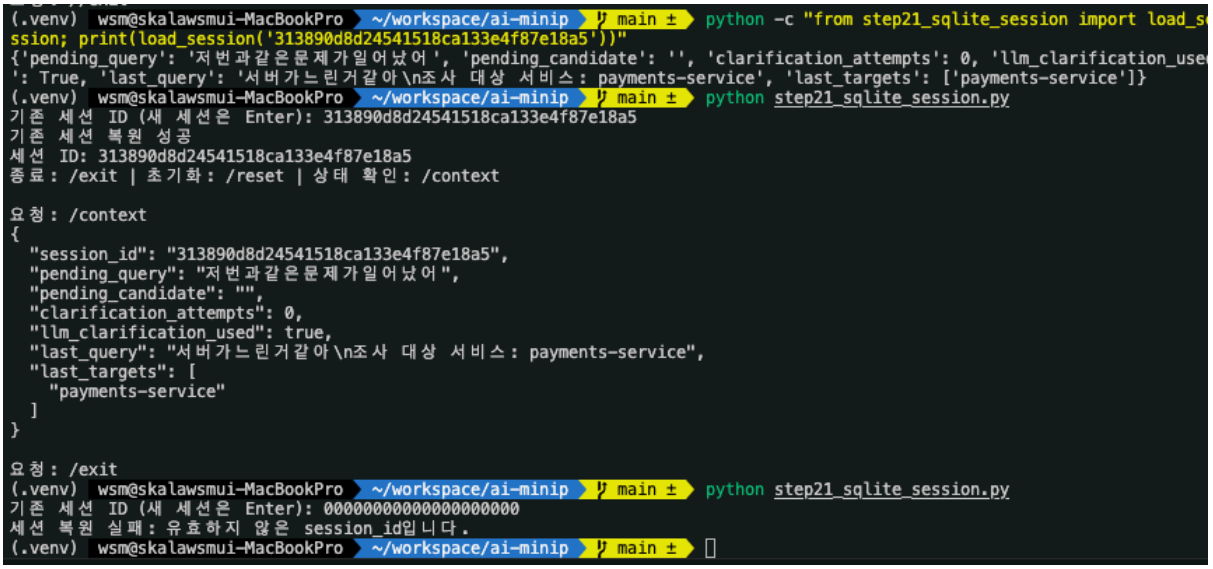
일부러 잘못된 서비스 이름을 작성해도 이제 fallback으로 재질문이 가능해졌다.
difflib.get_close_matches() 대신 llm fallback 재질문 route를 사용하고있는데
나중에 다른 시스템에 적용할 때는 정규식-closematch-llm fallback으로 이어지는 3중 입력처리도 가능하겠다는 생각을 했다.



Clarification 3회 제한을 확인하려고 봤는데, 첫 질문에서 pending query가 걸리지 않으면 Chat Route로 넘어가고 여기서 '지원하지않습니다' → END로 graph가 끝나고있다.

반복되는것은 While 루프의 CLI 입력일뿐으로, 이것 자체는 문제가 되지 않지만 'Route가 장애 요청을 일반 대화로 잘못 분류하는 경우'는 상당히 문제가 생길 수 있겠다는 insight를 얻었다.
만약에 이걸 처리해야한다고 했을때 Unclear라는 회색지대를 만들거나 뭔가 deterministic하게 요청을 받아야 할 것으로 보인다. 이건 개선과제로 남겨두려고 한다.

21 - 멀티 턴, SQLite 세션저장



21번은 많은 트러블 슈팅이 있었다. 따라서 이번 파트만 AI의 도움을 받아 정리했다.

STEP 21. 문제 발견 및 개선 결과

1. 원본 베이스 코드의 문제

멀티턴 맥락 관리 부족 - 이전 장애 증상과 조사 대상이 `plan()`에 제대로 전달되지 않음.
불명확한 서비스 처리 - 조사 대상이 명확하지 않아도 기본 서비스로 조회할 가능성이 있음.
세션 영속성 부족 - 실행 시 DB를 초기화하는 경로가 있어 프로그램 재시작 후 맥락 유지가 어려움.
Clarification 부재 - 조사 대상이 불명확할 때 재질문하고 후속 답변을 기존 요청과 연결하는 구조가 없음.

2. 우리 코드에서 새롭게 발생한 문제 (재작성 전 STEP 21 기준)

잘못된 세션 복원 처리 - 존재하지 않는 `session_id`를 입력해도 `load_session()`이 빈 State를 반환하여 정상 복원처럼 처리됨.
세션 복원 여부 확인 불가 - 세션 ID만 출력하고 실제 SQLite에 저장된 맥락을 확인할 방법이 없음.
세션 초기화 문제 - `/reset`에서 세션 행 자체를 삭제하므로 엄격한 세션 검증을 도입하면 기존 ID를 재사용할 수 없음.
후속 질문의 의도 손실 - 이전 서비스명은 기억하지만 새로운 질문의 분석 초점을 충분히 반영하지 못함.

3. 해결한 문제와 해결 방법

서비스 Hallucination 방지 - STEP 20에서 등록 서비스 목록 기반 검증과 Clarification을 추가하여 불명확한 서비스 조회를 차단함.
재질문 맥락 단절 해결 - `pending_query`에 이전 장애 요청을 저장하고, 사용자가 선택한 서비스와 결합하여 조사를 재개하도록 구현함.
세션 영속성 확보 - SQLite에 이전 장애 요청, 조사 대상, Clarification 상태를 저장하고 `session_id`로 복원하도록 구현함.
잘못된 세션 복원 방지 - 존재하지 않는 `session_id`를 입력하면 빈 State를 생성하는 대신 명시적으로 오류를 반환하도록 수정함.
세션 복원 검증 기능 추가 - `/context` 명령어로 SQLite에 저장된 실제 State를 확인할 수 있도록 구현함.
세션 초기화 문제 해결 - `/reset` 실행 시 세션 ID는 유지하고 저장된 State만 초기화하도록 변경함.
실행 검증 완료 - 프로그램 재시작 후 기존 `payments-service` 맥락 복원과 존재하지 않는 세션 ID의 오류 처리를 확인함.

4. 아직 해결하지 않은 문제

자연어 입력 패턴의 한계 - 정규표현식 기반 처리로 인해 `2번부터 확인해볼래`와 같은 문장형 선택을 인식하지 못하는 경우가 있음.
후속 질문의 세부 의도 반영 부족 - `그 서비스 메모리도 확인해줘`에서 조사 대상은 유지했지만 이전 CPU 장애 분석에 치중한 보고서가 생성됨.
전체 대화 기록 및 일반 대화 미지원 - 현재는 장애 조사에 필요한 요약 상태만 저장하므로 과거 대화를 자유롭게 회상하는 기능은 지원하지 않음.
사용자별 세션 접근 제어 부재 - 현재는 `session_id`만으로 세션에 접근할 수 있으며, STEP 23의 OAuth2 + JWT에서 인증 사용자와 세션의 소유 관계를 검증할 예정임.

[핵심 Insight]

Session Persistence와 Context Understanding은 별개의 문제다.
SQLite에 이전 대화의 상태를 정확히 저장하고 복원하더라도, Agent가 후속 질문의 의도를 올바르게 해석한다는 보장은 없다.
따라서 STEP 21에서는 저장·복원의 신뢰성을 우선 확보하고, 자연어 맥락 이해의 고도화는 별도 개선 과제로 분리했다.

22 - grounding

보고서의 내용이 문서의 내용과 맞는지 deterministic 하게 판단하는 코드를 작성했다.
Valid에서 semantic하게, 혹은 논리의 비약이 일어난 추론이 있는지 검사했다면 이번엔 아예 수치가 제대로 되었는지 검사한다.

이러한 코드처리가 가능한 이유는 원천데이터가 정형적인 기계의 로그이기 때문이다.

이러한 정형데이터에서 LLM 시스템이 필요한 이유가 무엇일까 잠깐 고민했고, 해당내용은 후술한다.

MCP에서 source를 가져올 때 출처 ID를 부여하고 보고서에 해당 출처ID를 첨부하도록 했다.

```
def verify_draft(draft: dict, sources: list[dict]) -> list[str]:
 errors = []
 if set(draft) != {"facts", "steps"}:
 errors.append("허용되지 않은 JSON 필드")
```

이런식으로 각 source들에 대해 ground 검증을 시도하는 코드를 작성했다.

23 - OAuth
fastAPI로 기능을 간단히 구현했다. LLM 관련 사항은 아니니 간단하게 줄인다.
OAuth2PasswordRequestForm (로그인 입력 처리) - 아이디·비밀번호를 /token으로 전달
OAuth2PasswordBearer (JWT 전달) - HTTP 요청의 Bearer Token 추출
Depends() (인증 자동 적용) - /chat 실행 전에 사용자 검증 함수 호출
Pydantic (입력값 검증 요청) - 데이터의 형식과 필수값 검사
HTTPException (오류 처리) - 인증 실패 401, 접근 거부 403 등 반환
Swagger UI (API 테스트) - /docs에서 로그인 및 보호된 API 테스트

3. 통합

README.md에 작성되었음.